

tetROS: Solving Automatic Tetris Assembly with a Modular Perception-Manipulation Pipeline

Rory O'Brien, Frank Zillmann, Moritz Cermann, Santiago Romero Ariaga

Problem Statement

Robotic assembly requires a robot to perceive an incomplete structure, identify missing components, and place them precisely, all autonomously.

Given a partially assembled Tetris-like grid and a nearby storage area of loose pieces, the robot must:

- Identify which pieces are missing from the grid
- Locate and grasp the correct piece from storage
- Transport and place it accurately while avoiding obstacles

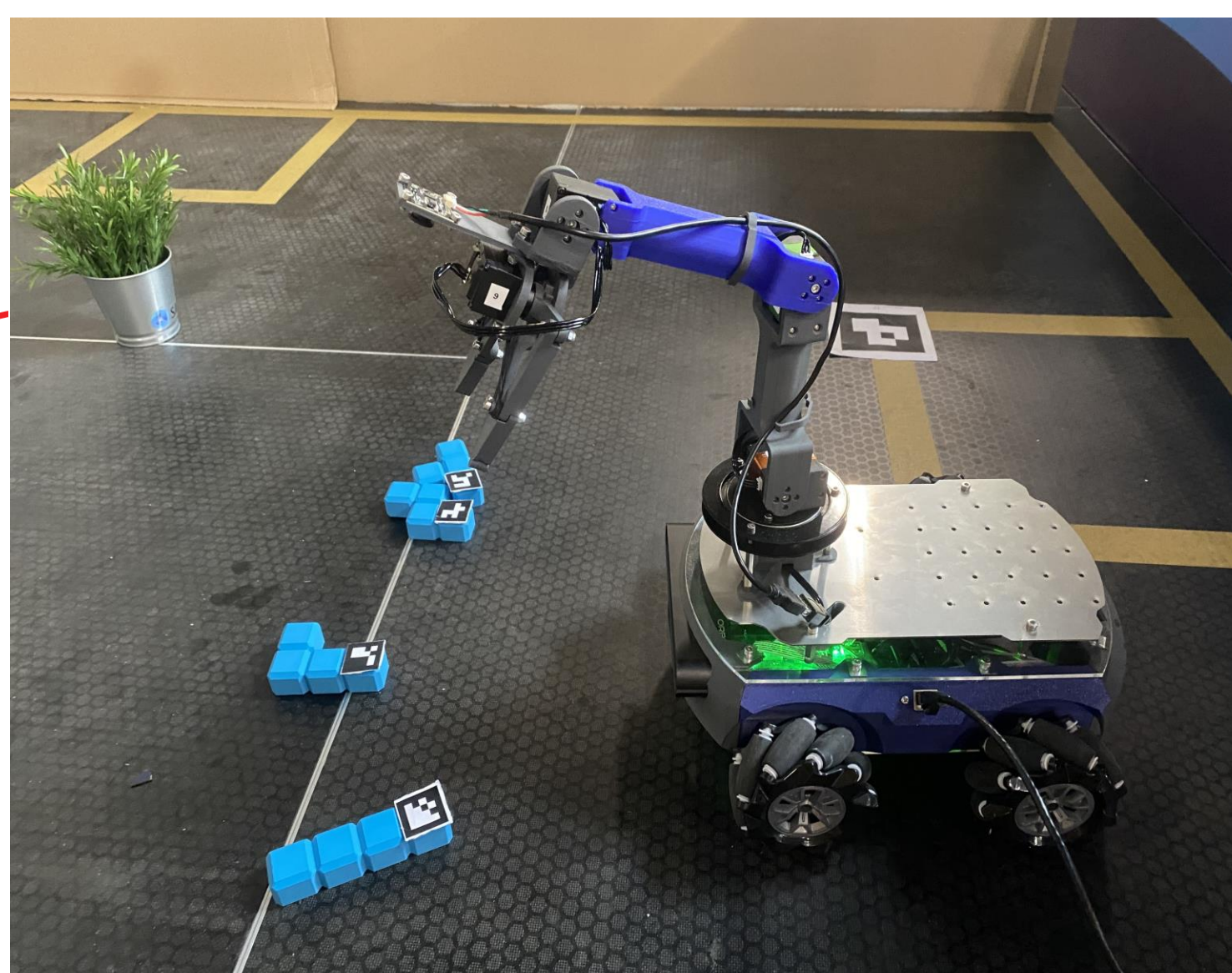
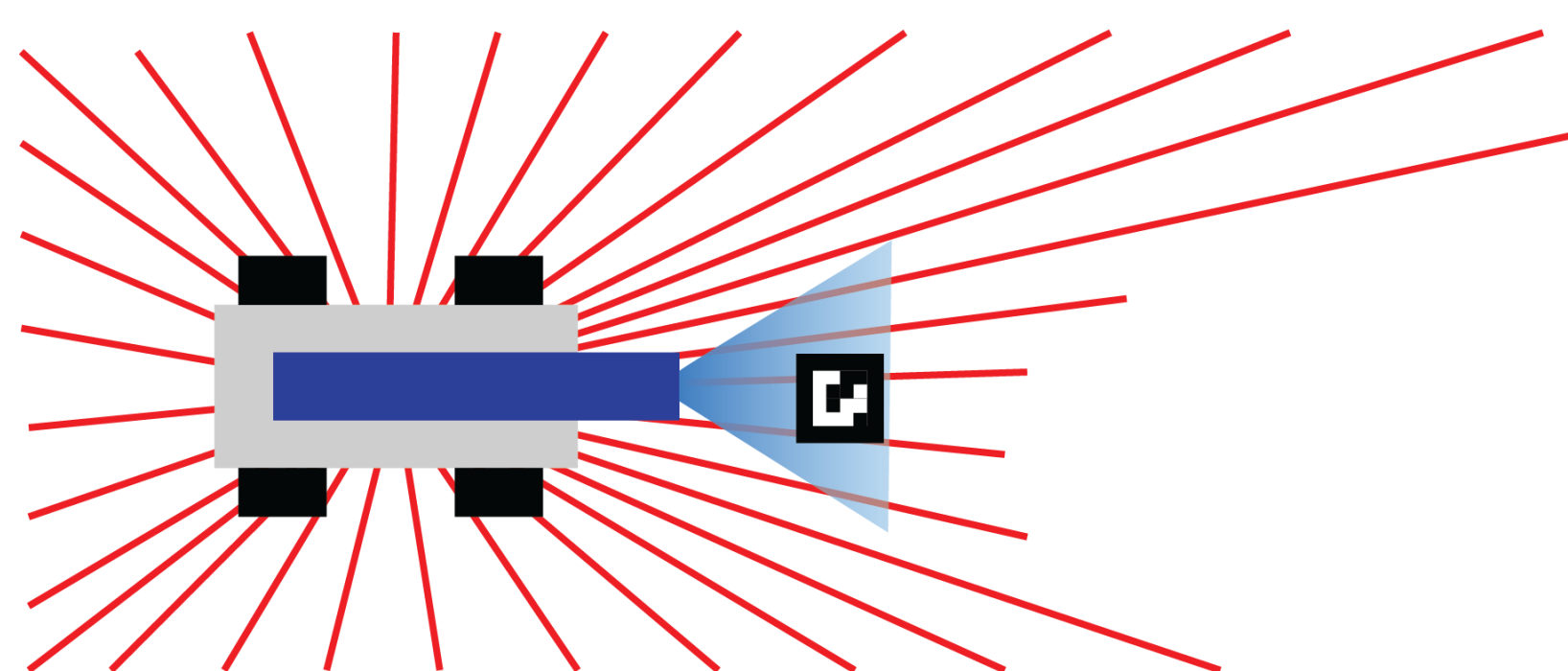
Methods

Integrate a full perception manipulation pipeline across four modules:

- **Mapping & Localisation:** LiDAR SLAM for map building and pose estimation
- **Grid Perception:** ArUco-based reference frame + HSV occupancy grid to detect missing pieces (good for finding position from far away)
- **Navigation:** Autonomous movement and obstacle avoidance via Nav2 (A* search for low computational intensity and optimal path)
- **Pick & Place:** IK-based grasp planning and execution via MoveIt on a 4-DOF arm (using C++ for better ROS integration)

Mapping and Localisation

- Occupancy map built offline via SLAM Toolbox with manual teleoperation, saved and reloaded at runtime
- Continuous localisation using SLAM Toolbox localisation mode
- ArUco markers at key locations used to establish reference frames for perception and navigation



The Robot

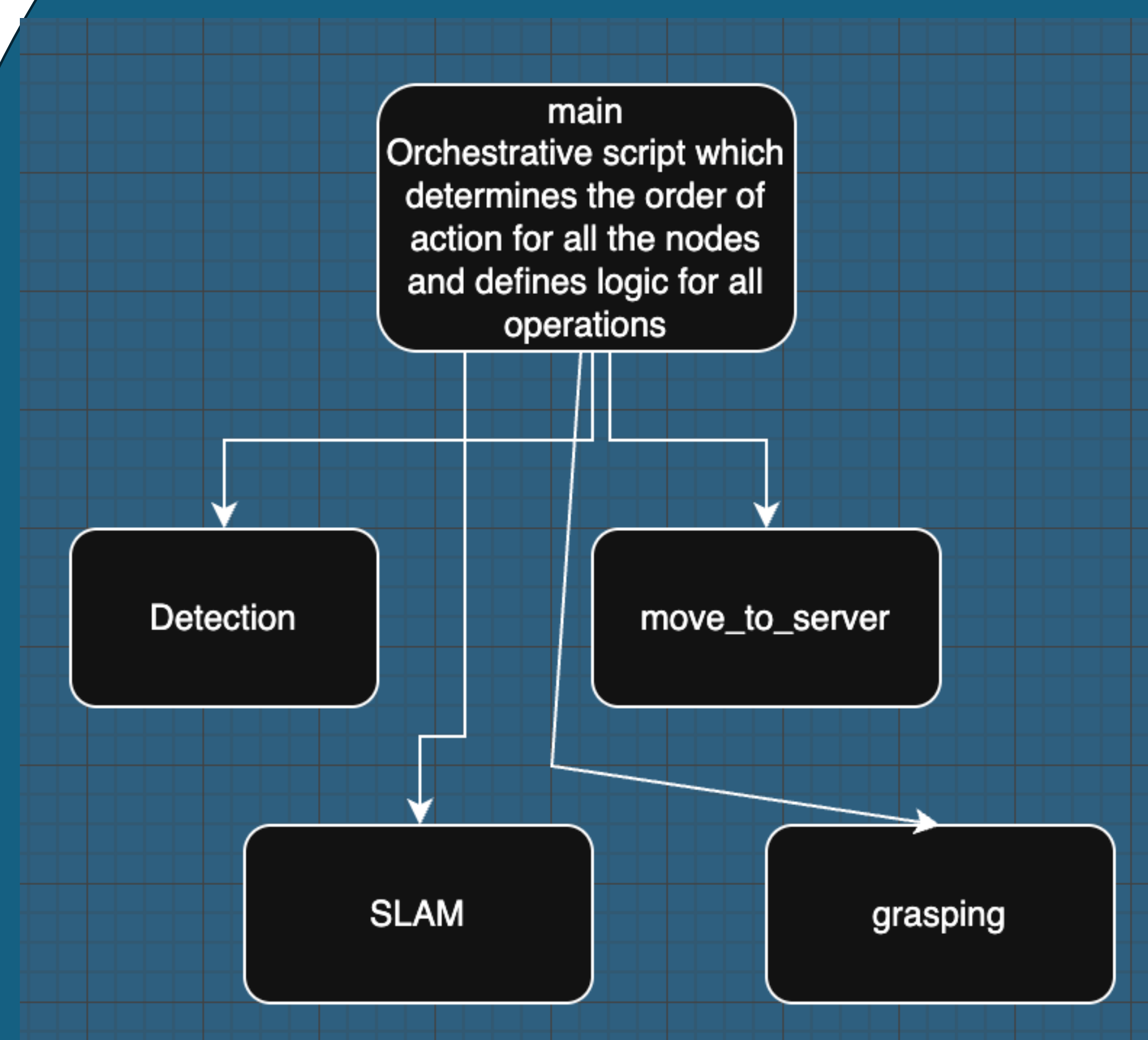


Sensors:
LiDAR – mapping
Laser scanners– mapping
Gripper camera – detection.
(Better than front camera because it can view markers parallel to the ground)
Wheel odometers – positioning

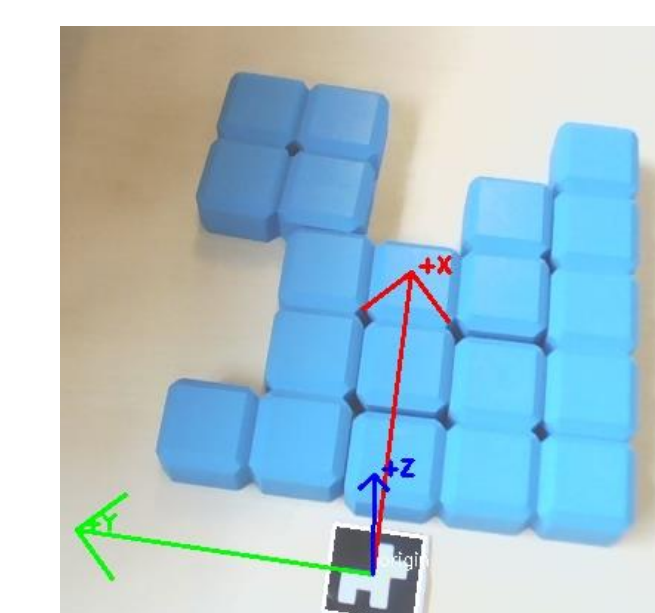


The robot is four wheel drive.
The gripper arm is three jointed with a panning at the base and bending joints at the 'shoulder', 'elbow', and 'wrist'.

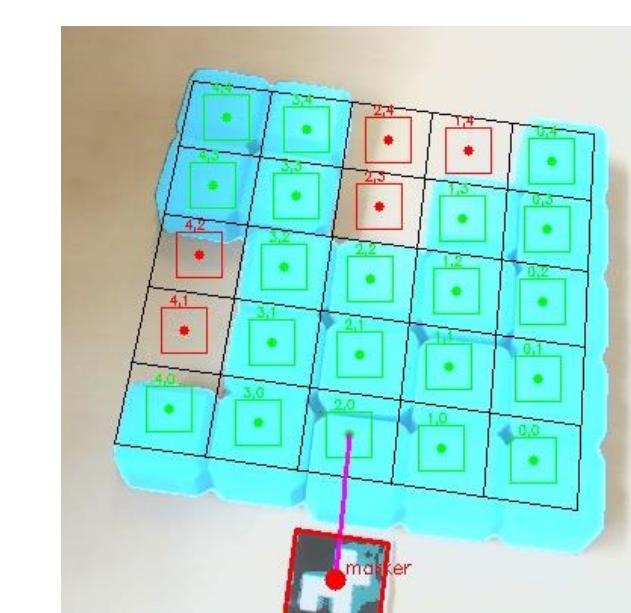
Simplified architecture



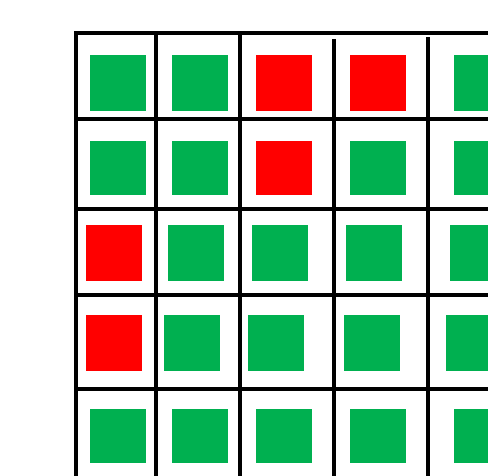
Grid Perception



ArUco Reference



Occupancy Grid



Binary Output



Tetris blocks

- Detect ArUco marker to establish coordinate frame and project cell boundaries into image space
- Each cell classified as occupied or empty via HSV colour thresholding
- Binary occupancy grid constructed from per-cell classification results

Navigation

- Goal poses obtained from marker detection and stored in move_to_server node
- Nav2 navigate_to_pose action used for point-to-point movement
- Global costmap built from the SLAM map
- local costmap updated live from LiDAR for dynamic obstacles
- Costmap inflation tuned to balance safety margins with navigability

Pick & Place

Pick-and-Place Process Using MoveIt

1. Detect tile position.
2. Validate reachability
3. Move gripper to just above the tile.
4. Lower vertically and grasp.
5. Move to a safe carry pose.
6. Move above the drop location.
7. Lower, release, and return to detection pose.

